

SOFTWARE CONSTRUCTION & DEVELOPMENT

Assignment 1 – Object-Oriented Programming

Name: Zoya Orakzai

Registration Number: 24ABSWE0008

Department: Software Engineering

Course Name: Software Construction & Development

Submitted to: Engr. Rizwan

GitHub Repository

<https://github.com/zoyercity/SC-Assignment-1-OOP>

Assignment 1

Task 1 – The 'Broken Vault' (Encapsulation)

Objective: Understand data hiding and why public fields are dangerous in software construction.

DigitalWallet.java

```
public class DigitalWallet {
    private String accountHolder;
    private double balance;
    private final String pinCode;

    public DigitalWallet(String accountHolder, double initialBalance, String pinCode) {
        this.accountHolder = accountHolder;

        if (initialBalance >= 0) {
            this.balance = initialBalance;
        } else {
            this.balance = 0;
        }

        this.pinCode = pinCode;
    }

    public boolean withdraw(double amount, String enteredPin) {
        if (enteredPin.equals(pinCode) && amount > 0 && amount <= balance) {
            balance -= amount;
            return true;
        }

        return false;
    }
}
```

Reflection

The original unencapsulated code could cause a catastrophic failure in a real banking application because any part of the program could directly modify a customer's balance. For example, a software bug could accidentally set an account's balance to a negative value or give a customer millions of extra money. Since the PIN was also public, unauthorized code could read or change a customer's PIN, creating a serious security risk.

Encapsulation prevents these problems by keeping sensitive data private and allowing it to be changed only through controlled methods.

Task 2 – The 'Evolving Workforce' (Inheritance & Polymorphism)

Employee.java

```
public class Employee {
    protected String name;
    protected double baseSalary;

    public Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    public double calculatePay() {
        return baseSalary;
    }
}
```

Developer.java

```
public class Developer extends Employee {
    private double techAllowance;

    public Developer(String name, double baseSalary, double techAllowance) {
        super(name, baseSalary);
        this.techAllowance = techAllowance;
    }
}
```

```

    }

    @Override
    public double calculatePay() {
        return baseSalary + techAllowance;
    }
}

```

SalesManager.java

```

public class SalesManager extends Employee {
    private double sales;

    public SalesManager(String name, double baseSalary, double sales) {
        super(name, baseSalary);
        this.sales = sales;
    }

    @Override
    public double calculatePay() {
        return baseSalary + (sales * 0.05);
    }
}

```

Main.java

```

import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<>();

        employees.add(new Developer("Ali", 80000, 15000));
        employees.add(new SalesManager("Sara", 60000, 200000));
        employees.add(new Developer("Ahmed", 75000, 10000));
        employees.add(new SalesManager("Ayesha", 55000, 150000));

        for (Employee employee : employees) {
            System.out.println(employee.name + " final pay: "
                               + employee.calculatePay());
        }
    }
}

```

Reflection

In my employees list, the variable employee is declared as an Employee, but the actual object can be a Developer or a SalesManager. When the loop reaches a Developer object, Java sees that the actual object is a Developer, so it uses the Developer version of calculatePay() and adds the tech allowance. When it reaches a SalesManager, Java uses the SalesManager version instead and calculates the commission from the sales. This happens at runtime, which allows the same employee.calculatePay() statement to produce different results depending on the actual object stored in the list.

Task 3 – 'Design by Contract' (Abstraction)

SmartDevice.java

```

public interface SmartDevice {
    void turnOn();
    void turnOff();
    String getStatus();
}

```

SmartBulb.java

```

public class SmartBulb implements SmartDevice {

    private boolean isOn;
    private int brightness;
}

```

```

public SmartBulb() {
    isOn = false;
    brightness = 50;
}

@Override
public void turnOn() {
    isOn = true;
}

@Override
public void turnOff() {
    isOn = false;
}

@Override
public String getStatus() {
    if (isOn) {
        return "Smart Bulb is ON at " + brightness + "% brightness.";
    } else {
        return "Smart Bulb is OFF.";
    }
}

public void setBrightness(int level) {
    if (level >= 0 && level <= 100) {
        brightness = level;
    }
}
}

```

SmartThermostat.java

```

public class SmartThermostat implements SmartDevice {

    private boolean isOn;
    private double temperature;

    public SmartThermostat() {
        isOn = false;
        temperature = 22.0;
    }

    @Override
    public void turnOn() {
        isOn = true;
    }

    @Override
    public void turnOff() {
        isOn = false;
    }

    @Override
    public String getStatus() {
        if (isOn) {
            return "Smart Thermostat is ON at " + temperature + "°C.";
        } else {
            return "Smart Thermostat is OFF.";
        }
    }

    public void setTemperature(double temp) {
        if (temp >= 10 && temp <= 35) {
            temperature = temp;
        }
    }
}

```

Reflection

We used an interface because a SmartBulb and a SmartThermostat are different types of devices, but they still need to provide the same basic behaviors. The SmartDevice interface forces both classes to implement turnOn(), turnOff(), and getStatus(), while allowing each device to implement these methods in its own way. This gives the software architect a clear contract that every new smart device must follow. It also makes the system easier to expand because new devices can implement SmartDevice without needing to inherit from one specific

parent class.

Task 4 – The AI Code Review (Meta-Learning)

Prompt given to AI: “Write a Java program for a simple Library System using OOP. Include classes for Book and Member.”

Original AI-generated code

```
class Book {
    String title;
    String author;
    boolean isAvailable;

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
        this.isAvailable = true;
    }

    public void borrowBook() {
        if (isAvailable) {
            isAvailable = false;
            System.out.println(title + " has been borrowed.");
        } else {
            System.out.println(title + " is not available.");
        }
    }

    public void returnBook() {
        isAvailable = true;
        System.out.println(title + " has been returned.");
    }
}

class Member {
    String name;
    int memberId;

    public Member(String name, int memberId) {
        this.name = name;
        this.memberId = memberId;
    }

    public void displayMemberInfo() {
        System.out.println("Member: " + name);
        System.out.println("Member ID: " + memberId);
    }
}

public class LibrarySystem {
    public static void main(String[] args) {

        Book book1 = new Book("The Hobbit", "J.R.R. Tolkien");
        Member member1 = new Member("Ali", 101);

        member1.displayMemberInfo();

        book1.borrowBook();
        book1.returnBook();
    }
}
```

One thing the AI did well

The AI used constructors in both the Book and Member classes to initialize their objects. For example, the Book constructor receives a title and author and also sets the initial availability. This is a good OOP practice because the object is initialized when it is created.

Design flaw identified

The fields in both classes were declared without the private access modifier. This means other classes could directly change important data such as isAvailable, name, or memberId. For example, external code could directly set book1.isAvailable to false without using borrowBook(). This weakens encapsulation because the

class does not control how its internal data is modified.

Corrected Book.java

```
public class Book {
    private String title;
    private String author;
    private boolean isAvailable;

    public Book(String title, String author) {
        this.title = title;
        this.author = author;
        this.isAvailable = true;
    }

    public void borrowBook() {
        if (isAvailable) {
            isAvailable = false;
            System.out.println(title + " has been borrowed.");
        } else {
            System.out.println(title + " is not available.");
        }
    }

    public void returnBook() {
        isAvailable = true;
        System.out.println(title + " has been returned.");
    }
}
```

Note: The original AI code is included here because Task 4 specifically asks for the original output, critique, and refactored code.